FYP ROLE DISTRIBUTION

Smart Academic Communication Platform - Roles

MAY 23, 2026

Table of Contents

1. Recommended System Architecture 4
1.1 Core Architecture (.NET) 4
1.1.1 Handled by: 4
1.1.2 Responsibilities: 4
1.1.3 Recommended: 4
1.2 Python AI/ML Service 5
1.2.1 Handled by: 5
1.2.2 Responsibilities: 5
1.2.3 Recommended: 5
1.2.4 Communication: 5
1.3 React Frontend 5
1.3.1 Handled by: 5
1.3.2 Responsibilities: 6
1.3.3 Recommended: 6
1.4 Node.js Supporting Service Layer 6
1.4.1 Handled by: 6
1.4.2 Responsibilities: 6
1.4.3 Recommended: 7
1.4.4 Outcomes 7
2. Detailed Role Distribution 7
2.1 Talha — Technical Lead & Core Architect 7
2.1.1 System Architecture 7
2.1.2 Core Backend Development 7
2.1.3 DevOps & Standards 8
2.1.4 Integration Ownership 8
2.2 Raza — Frontend Lead 8
2.2.1 Frontend Architecture 8
2.2.2 UI/UX Implementation 8

2.2.3 API Integration 9
2.2.4 Visualization 9
2.2.5 Additional Ownership 9
2.3 Areeb — AI/ML & Intelligent Analytics 9
2.3.1 ML System Development 9
2.3.2 Intelligent Features 9
2.3.3 Data Processing 10
2.3.4 Python AI Service 10
2.3.5 Analytics Support 10
2.4 Furqan — Full Stack Support & Real-Time Systems 10
2.4.1 Backend Support 10
2.4.2 Real-Time Communication Module 10
2.4.3 File & Document Sharing 11
2.4.4 Frontend Support 11
2.4.5 Optional Node.js Microservice 11
3. Suggested Module Ownership Table 11
4. Best Technical Distribution 12
4.1 Recommended Tech Stack 12
4.2 Important Architectural Recommendation 12
4.2.1 Principle 12
4.2.2 Keep in .NET 13
4.2.3 Use Node.js ONLY for 13
5. Final Suggested Team Workflow 13
5.1 Phase 1 — Architecture & Database 13
5.1.1 Lead: 13
5.1.2 Support: 13
5.1.3 Deliverables: 14
5.2 Phase 2 — Core Backend + Frontend Foundation 14
5.2.1 Parallel Work: 14

5.3 Phase 3 — AI + Automation 14
5.3.1 Parallel Work: 14
5.4 Phase 4 — Integration 14
5.4.1 Lead: 14
5.4.2 Support: 14
5.4.3 Tasks: 14
6. Additional Features (Optional) 15

1. Recommended System Architecture

1.1 Core Architecture (.NET)

1.1.1 Handled by:

- Talha Ahmad (Lead)
- Furqan (Support)

1.1.2 Responsibilities:

- Main business logic
- Shared services
- Central database access
- Authentication & authorization
- Academic workflows
- Rule engine
- Campaign management
- Notification orchestration
- Analytics APIs
- Shared file services
- Background jobs
- API gateway/core APIs

1.1.3 Recommended:

- ASP.NET Core Web API
- Clean Architecture
- JWT/Auth
- SignalR (if kept inside .NET)
- Hangfire/Quartz for scheduling
- SQLite/PostgreSQL/MySQL

1.2 Python AI/ML Service

1.2.1 Handled by:

- Areeb

1.2.2 Responsibilities:

- Email classification
- Priority prediction
- Spam filtering
- Future analytics intelligence
- NLP preprocessing
- Recommendation logic
- Smart communication analysis

1.2.3 Recommended:

- FastAPI/Flask
- Scikit-learn
- Pandas
- NLP libraries
- Separate microservice communicating with .NET APIs

1.2.4 Communication:

- REST API between .NET and Python
- Shared centralized database (read-controlled)
- Queue-based communication optional

1.3 React Frontend

1.3.1 Handled by:

- Raza (Lead)
- Furqan (Support)

1.3.2 Responsibilities:

- Full admin dashboard
- Faculty/student UI
- Analytics visualization
- Campaign creation UI
- Rule builder UI
- Real-time chat UI
- Notification UI
- Responsive layouts
- API integration

1.3.3 Recommended:

- React + TailwindCSS
- Redux/Zustand
- Chart.js/Recharts
- Socket integration

1.4 Node.js Supporting Service Layer

1.4.1 Handled by:

- Furqan

1.4.2 Responsibilities:

- Real-time messaging optimization
- WebSocket scaling support
- File upload processing
- Media/document handling
- Optional notification broker
- Optional queue/event processing
- Chat microservice (if separated)

1.4.3 Recommended:

- Node.js + Express
- Socket.IO
- Redis (optional)

1.4.4 Outcomes

- Core business system in .NET
- AI in Python
- High-concurrency communication tasks optionally in Node.js

That division is very practical.

2. Detailed Role Distribution

2.1 Talha — Technical Lead & Core Architect

2.1.1 System Architecture

- Define full system architecture
- Database schema design
- API standards
- Security design
- Service communication design
- Integration strategy

2.1.2 Core Backend Development

- Authentication & authorization
- Academic Data Management APIs
- Rule Engine
- Campaign engine
- Notification orchestration
- Shared services

- Scheduling/background jobs
- Communication tracking
- Logging system
- Analytics APIs

2.1.3 DevOps & Standards

- Git workflow
- Project structure
- Backend coding standards
- API documentation
- Integration supervision

2.1.4 Integration Ownership

You should own:

- Final integration of all modules
- Deployment architecture
- Inter-service communication

2.2 Raza — Frontend Lead

2.2.1 Frontend Architecture

- React project structure
- Routing
- State management
- Component design system

2.2.2 UI/UX Implementation

- Dashboard pages
- Analytics UI
- Automation builder UI
- Campaign management UI

- Template editor UI
- Chat interface
- Notification center

2.2.3 API Integration

- Connect all frontend modules with backend APIs
- Authentication flow integration
- Real-time frontend socket integration

2.2.4 Visualization

- Charts
- Graphs
- Reports
- Communication analytics

2.2.5 Additional Ownership

- Responsive design
- Tailwind component system
- Frontend optimization

2.3 Areeb — AI/ML & Intelligent Analytics

2.3.1 ML System Development

- Email classification model
- Spam detection
- Priority ranking
- Message categorization

2.3.2 Intelligent Features

- Smart communication suggestions
- Personalized messaging logic
- Context-aware communication scoring

- Behavioral analysis

2.3.3 Data Processing

- Dataset preparation
- Model training
- Feature engineering
- Evaluation metrics

2.3.4 Python AI Service

- AI API endpoints
- ML inference service
- Integration with .NET backend

2.3.5 Analytics Support

- Predictive insights
- Future trend analysis
- Communication efficiency scoring

2.4 Furqan — Full Stack Support & Real-Time Systems

2.4.1 Backend Support

Support Talha in:

- API implementation
- CRUD modules discussion
- Notification services planning
- File/document modules planning
- Shared backend utilities planning

2.4.2 Real-Time Communication Module

- Chat backend
- Socket handling
- Presence system

- Message delivery state
- Typing indicators
- Real-time notifications

2.4.3 File & Document Sharing

- Upload system
- File validation
- Media handling
- Download APIs

2.4.4 Frontend Support

Support Raza in:

- React integration
- Real-time UI
- Chat frontend
- Notification UI

2.4.5 Optional Node.js Microservice

If we separate chat:

- Socket.IO server
- Redis pub/sub
- Real-time scaling layer

3. Suggested Module Ownership Table

MODULE	PRIMARY OWNER	SECONDARY SUPPORT
AUTHENTICATION & AUTHORIZATION	Talha	Furqan
ACADEMIC DATA MANAGEMENT	Talha	Furqan
TEMPLATE MANAGEMENT	Talha	Raza
CAMPAIGN MANAGEMENT	Talha	Raza

AUTOMATION RULE ENGINE	Talha	Areeb
MESSAGING SERVICES	Talha	Furqan
REAL-TIME CHAT	Furqan	Raza
EMAIL CLASSIFICATION ML	Areeb	Talha
ANALYTICS DASHBOARD	Raza	Areeb
REPORTING API	Talha	Raza
NOTIFICATION SYSTEM	Talha	Furqan
FILE SHARING MODULE	Furqan	Talha
FRONTEND UI/UX	Raza	Furqan
AI INTEGRATION	Areeb	Talha
DEPLOYMENT & INTEGRATION	Talha	All

4. Best Technical Distribution

4.1 Recommended Tech Stack

Layer	*Technology*
Main Backend	ASP.NET Core
AI Service	Python FastAPI
Frontend	React + Tailwind
Real-Time Layer	SignalR OR Socket.IO
Database	PostgreSQL/AzureSQL
Background Jobs	Hangfire
Cache/Queue	Redis (optional)

4.2 Important Architectural Recommendation

4.2.1 Principle

Do NOT split core business logic into:

- .NET backend
- Node backend

That creates:

- duplicated logic
- inconsistent security
- maintenance complexity

Instead:

4.2.2 Keep in .NET

- ALL core workflows
- Database ownership
- Security
- Automation engine
- Analytics APIs
- Shared services

4.2.3 Use Node.js ONLY for

- Real-time communication
- High-concurrency socket handling
- Optional media processing

This is the cleanest approach academically and technically.

5. Final Suggested Team Workflow

5.1 Phase 1 — Architecture & Database

5.1.1 Lead:

- Talha

5.1.2 Support:

- Furqan

5.1.3 Deliverables:

- ER Diagrams
- API structure
- Service contracts
- Database schema

5.2 Phase 2 — Core Backend + Frontend Foundation

5.2.1 Parallel Work:

- Talha → Core APIs
- Raza → React UI
- Furqan → Supporting APIs
- Areeb → Dataset preparation

5.3 Phase 3 — AI + Automation

5.3.1 Parallel Work:

- Areeb → ML model
- Talha → Rule engine
- Raza → Analytics dashboards
- Furqan → Real-time system

5.4 Phase 4 — Integration

5.4.1 Lead:

- Talha

5.4.2 Support:

- All

5.4.3 Tasks:

- Service integration
- Deployment

- Testing
- Optimization
- Final debugging

6. Additional Features (Optional)

These would make your FYP significantly stronger without becoming unrealistically complex:

1. Smart communication priority scoring
2. AI-generated communication summaries
3. Student risk prediction (attendance/performance)
4. Automated escalation workflows
5. Smart notification batching
6. Communication sentiment analysis
7. Faculty workload analytics
8. Read receipt analytics
9. AI-generated campaign recommendations
10. Predictive engagement analytics